CYBERALCHEMY TECHNICAL OVERVIEW

From Business Intent to Bounded Evidence

A governed path through definitions, ontologies, FormalClaims, AlchLang, DomainSpec, TDE, formal providers, and EVF

Preserve meaning across the entire construction chain—and never allow a claim to become stronger than the evidence supporting it.

Prepared by Vlad

Discussion draft · 20 September 2026

EXECUTIVE OVERVIEW

The continuity problem

Teams usually lose intent not in one dramatic failure, but through a sequence of plausible translations: requirement to architecture, architecture to code, code to tests, and tests to claims.

Meaning drifts

The same word acquires different meanings across product, engineering, and formal models.

Evidence inflates

A passing test, solver result, or proof is allowed to imply more than its exact model and observation justify.

Authority blurs

Generated artifacts and technical checks are treated as if they could approve meaning, implementation, or release.

CyberAlchemy is an attempt to preserve an inspectable chain from the words people use to the bounded conclusions engineers may honestly make.

Definitions → **Business ontology** → **AlchLang** → **DomainSpec graph** → **Evidence** → **Owner decision**

The central separation

Question	Responsible layer	What it must not silently decide
What do the words mean?	Definitions and domain owners	Implementation or proof
How are meanings connected?	Business ontology	Canonical truth merely by being a graph
What exact proposition is asserted?	FormalClaim	Whether the proposition is desirable or implemented
What software semantics are declared?	AlchLang and DomainSpec source	Provider selection or production behavior
What did a tool observe?	TDE route or formal provider	Business acceptance or universal correctness
What conclusion does the evidence support?	EVF	Approval, deployment, or authority

Claim ceiling. This document explains a product architecture and its current evidence boundaries. It does not claim that every intended connection is complete, formally proven, production-validated, or externally released.

THE COMPLETE PATH

Ten bounded stages, not one magical compiler

CyberAlchemy is deliberately a composition of languages, contracts, projections, providers, and gates. Each stage answers one question and preserves a reference to the stage before it.

1	**Define the domain.** Clarify goal, intent, context, concepts, evidence, owners, and open questions.	Define
2	**Govern the vocabulary.** Reuse, specialize, propose, or leave each important definition open.	Definition owners
3	**Map business meaning.** Connect typed meaning nodes with provenance, lifecycle, confidence, residue, and challenge paths.	Business ontology
4	**State exact propositions.** Bind selected business meaning to FormalClaims with explicit signatures and ceilings.	Domain + formalization
5	**Author software semantics.** Express domain roles, relationships, integration intent, and selected profiles through AlchLang.	AlchLang
6	**Compile the semantic graph.** Validate types and relationships; emit deterministic graph, obligations, projections, manifests, and receipts.	DomainSpec
7	**Realize the system.** Select target language, architecture realization, operational providers, and configuration slots.	Runtime profiles
8	**Derive bounded checks.** Produce provider-neutral obligations and cases for an admitted executable slice.	TDE
9	**Collect exact evidence.** Run tests, solvers, model checkers, proof assistants, observers, or runtime harnesses under explicit bounds.	Evidence providers
10	**Join without inflation.** Map current evidence to exact obligations and expose unsupported, stale, missing, or contradictory residue.	EVF + owner gate

> **Why this composition matters.** A TLA+ model, an Alloy exploration, a Lean theorem, a generated test, and a runtime observation answer different questions. CyberAlchemy preserves their identities instead of turning them into interchangeable “verification.”

STAGES 1–3

Definitions and business ontology

Formal precision cannot repair a term whose owner, scope, or intended use was never settled.

Definition Pack

The Define process starts from a real business question and records the minimum semantic context required for downstream work:

What is being attempted?

Goal, original intent, bounded context, intended consumers, constraints, and desired outcome.

What must be understood?

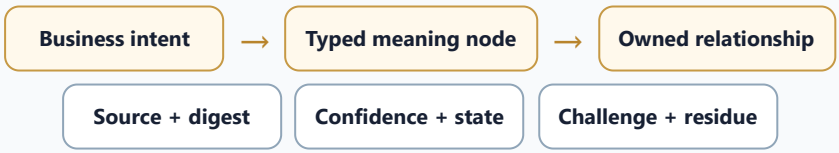
Concepts, exact sources, supporting evidence, meaning owners, and unresolved questions.

Every concept receives a visible disposition

Disposition	Meaning	Why it matters
Reuse	An existing owned definition fits this context.	Prevents needless synonyms.
Specialize	An existing definition fits after narrowing its context.	Makes contextual differences explicit.
Propose	A new candidate definition is required.	Separates invention from accepted meaning.
Open	The answer is not ready.	Prevents ambiguity from becoming an implementation guess.

From vocabulary to ontology

A business ontology is not merely a collection of nouns. It is a governed meaning layer:



Each important ontology record can carry provenance, branch or role, confidence, candidate or promoted state, drift handling, and downstream-use obligations. Business, system, and bridge branches remain separate so that implementation evidence cannot silently redefine business intent.

> **Non-collapse rule.** An ontology is a navigable and governed map of meaning. Its existence does not make every node canonical, prove a system exists, or authorize implementation.

STAGE 4

FormalClaim: a precise bridge, not a proof

A FormalClaim freezes the meaning-bearing inputs of one proposition while keeping evaluation context, provider syntax, and evidence outside the claim's identity.

```
FormalClaim = (
  business meaning,
  explicit accepted-definition closure,
  formal signature,
  canonical proposition,
  meaning-bearing language profile,
  assumptions and scope,
  claim ceiling,
  deterministic identity
)
```

Example: order submission eligibility

Business voice

"An order may be submitted only if it has at least one line and its total is non-negative."

Formal voice

```
∀ o : Order ·
  HasLine(o) ∧
  HasNonNegativeTotal(o)
```

Domain-context voice

This proposition governs submission eligibility. It does not claim that an order was persisted, a message was delivered, payment was authorized, or a production system is correct.

One claim, several formal routes

Profile	Question it can address	Bound that remains
FOL / sorted FOL	Precise predicates over an explicit signature	A language, not an executor
SMT	Satisfiability or validity of an admitted projection	Unknown and timeout retain their native meaning
TLA+/TLC	Temporal behavior across a declared state machine	Model checking is bounded by model and explored state space
Alloy	Relational instances and counterexamples	Bounded by the exact scope
Lean	Theorem checking in an exact formal environment	Does not show the running implementation preserves the model
Soufflé	Finite closed-world derivation	Depends on exact facts and rules

Human explainability. The system does not "translate a formula back into truth." It creates an understanding view made of source-linked ontology nodes—BusinessMeaning, definition binding, signature, proposition, assumptions, ceiling, dependencies, projections, and evidence. Each node explains one part while the original FormalClaim retains semantic identity.

THE UNDERSTANDING LAYER

Store as a graph. Explain as a sequence.

DomainSpec preserves exact claim structure as a graph, but presents the explanation in the order a person can follow: context, formal source, operator, operands, definitions, recomposition, and boundary.

Two identities and two shapes remain separate

Source graph

`FormalClaim → clause → expression → predicate → definition`. Stable identities and typed edges preserve exact provenance and machine navigation.

Serial understanding projection

The same nodes receive a declared reading order. Provenance sits in a secondary rail, so presentation becomes linear without changing graph semantics.

Worked reading spine: one expression, step by step

```
FormalClaim: FC-OrderSubmission-v1 · clause 3
Eligible(o) ∧ ValidTotal(o)
```

1	**Orient, then read the source.** We are deciding whether an `Order` satisfies the submission conditions. Preserve the exact clause before explaining it.	context + source
2	**Explain the operator.** ∧ means both conditions must hold at the same time.	expression root
3	**Read the left operand.** `Eligible(o)` means the Order satisfies `EligibleOrder@2`, whose formal, plain-language, and domain-context voices are linked.	predicate 1 + definition
4	**Read the right operand.** `ValidTotal(o)` means the total satisfies `ValidTotal@1`, again exposing its three definition voices.	predicate 2 + definition
5	**Recompose.** The Order must be eligible and its total valid; neither condition is sufficient alone.	whole meaning
6	**Close the boundary.** This sequence explains clause 3. It cannot replace or modify it.	authority

Provenance rail attached to the selected explanation

Understanding node

```
Node: U-FC-ORDER-003-CONJUNCTION
Origin: structurally-derived
Source: clause 3, expression root
Definitions: EligibleOrder@2, ValidTotal@1
Authority: explanatory-only
```

Required behavior

The node attaches to the exact conjunction. Its definition links come only from explicit `uses_definition` edges. Missing selectors fail closed. Screen position cannot create meaning.

DomainSpec candidate boundary. `FormalClaimUnderstandingProjection/v1-candidate` records the graph, selectors, definition links, serial order, recomposition, and provenance rail. The contract is fixture-validated but not yet compiler-emitted. Its nodes are explanatory only: they cannot modify the claim, redefine terms, prove comprehension, or establish runtime behavior.

STAGES 5–6

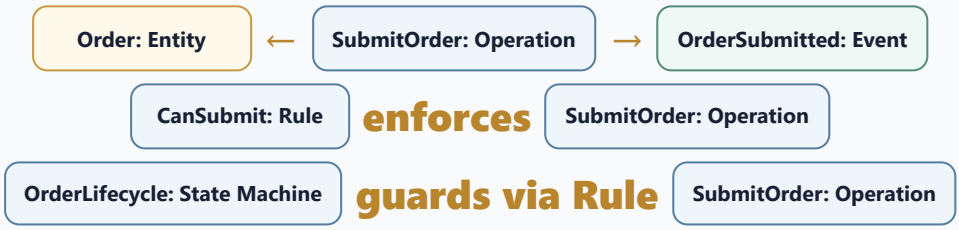
AlchLang and the DomainSpec semantic graph

AlchLang is the human-facing authoring language. DomainSpec owns the canonical typed graph, relationship constraints, compilation semantics, and evidence boundaries.

The thirteen active backend semantic roles

Family	Meta-types	Question answered
Structural	Entity · Value Object · Enum	What things and values exist?
Behavioral	Operation · Query · Calculation · Rule · Policy · Workflow	What happens, is computed, or is decided?
Connective	Interface · Event · Mapping	How do parts communicate or transform information?
Lifecycle	State Machine	How does a concept move between declared states?

Relationships have typed consequences



The edge is not decorative. Its source type, target type, and consequence are checked. A Rule can gate an Operation; an Operation can produce an Event; a Workflow can orchestrate Operations. Arbitrary combinations are rejected or remain unsupported rather than being guessed.

Provider-neutral integration intent

What source may say

Store Order; publish OrderSubmitted; consume a message; invoke an Operation; call an external API; acknowledge after application commit.

What source does not imply

PostgreSQL, NATS, HTTP framework, dependency version, credential, endpoint, deployment platform, or production approval.

Current boundary. The concise syntax shown in the authoring guide is partly target syntax. Underlying JSON contracts, compiler paths, graph construction, and substantial lowering exist, but not every concise form is accepted by the current parser.

REUSABLE RIGOR

What is proved once—and what must be re-established

Generated code does not begin from an unexamined template. It begins from admitted, versioned realization contracts—but reusable evidence must not be confused with proof of a new application.

The reusable construction basis

Formalized roles

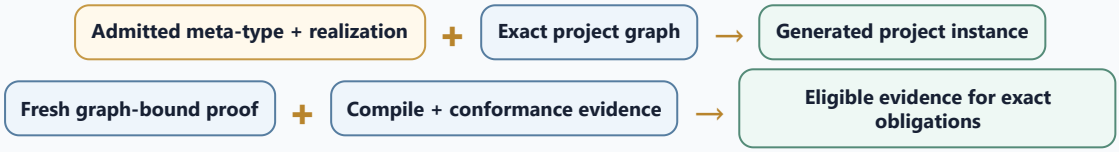
Each meta-type has machine-checkable formation criteria and typed edge-participation rules. The active backend language contains 13 roles and 31 governed relationship signatures.

Admitted realizations

Each active role has one selected TypeScript, Python, and Rust realization: 39 exact, digest-bound realization rows exercised by deterministic generation and bounded static conformance.

Graph-bound proof

For an exact compiled project, DomainSpec can generate a Lean proof unit covering applicable DS-D8 graph obligations and selected L1 structural facts, then check the resulting certificate independently.



What the generated project can inherit

Reusable evidence	What it can support	What it cannot establish
Meta-type criteria and relationship signatures	The authored nodes and edges are well formed under the selected language contract.	That the business meaning is correct or complete.
Exact realization row and deterministic generator	The artifact was produced from the selected versioned realization and can be reproduced.	That arbitrary edits or a different environment preserve the realization.
Generated Lean proof unit	Applicable structural propositions hold for the exact compiled graph and proof binding.	Runtime behavior, external-provider behavior, or production correspondence.
Compile and static-conformance checks	The exact generated artifacts compile and match their selected contracts at the observed boundary.	All executions, failure modes, integrations, or operational paths.

The defensible advantage. A generated project starts from a pre-qualified, content-bound construction basis. Applicable structural obligations can be discharged by fresh replay of the reusable proof and conformance machinery instead of being rediscovered manually.

Not an automatic EVF pass. EVF may count only evidence bound to the new project's exact source, graph, realization rows, generated bytes, assumptions, provider, environment, and obligation IDs. New composition, integration, runtime, and business obligations still require their own current evidence. The result, when the required join closes, is `PASS_BOUNDED` for named obligations—not "all generated code is proven."

STAGES 6–7

Compilation, system ontology, and realization

DomainSpec compilation answers a concrete question: “Given this exact authored model, what system structure follows from it, what must a realization provide, and how can we identify the result?”

Compilation in plain language

Input

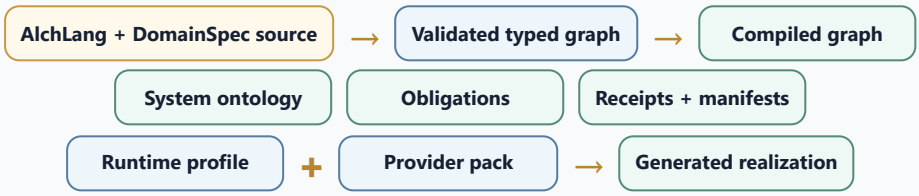
Exact AlchLang and DomainSpec declarations: typed concepts, allowed relationships, FormalClaim bindings, integration intent, and profile selections.

Work

Resolve names, reject missing or invalid endpoints, check type–relationship compatibility, select exact realization contracts, and derive required obligations.

Output

A content-identified compiled graph, readable projections, system ontology, implementation plan or artifacts, and receipts binding all outputs to their inputs.



Why two graph products?

Artifact	Purpose	Boundary
Compiled graph	Resolved machine representation used by compilers and validators	Does not establish runtime behavior
System ontology	Traceability view across source concepts, system roles, evidence, and residue	Does not redefine source or admit the system
Obligations	Target-neutral statements of what a realization or observer must establish	Are not evidence of satisfaction
Receipt	Digest-bound record of exact inputs, tool versions, and outputs	Is not approval or conformance by itself

A concrete compiler example

```
Authored source:
  Rule OrderCanBeSubmitted --enforces--> Operation SubmitOrder
  Operation SubmitOrder --produces--> Event OrderSubmitted

DomainSpec checks:
  • both referenced nodes exist
  • “enforces” accepts Rule → Operation
  • “produces” accepts Operation → Event
  • the selected target has realizations for all three roles

Compiler products:
  • graph: exact nodes and typed edges
  • obligation: SubmitOrder must be guarded by OrderCanBeSubmitted
  • system ontology: trace from business nodes to system roles and residue
  • receipt: hashes of source, profile, compiler, graph, and generated output
```

The compiled graph is the machine’s resolved model. The system ontology is the inspectable traceability projection of that model. Generated code is one selected realization. None of these is allowed to contradict or independently redefine the authored source.

Realization remains explicit

A runtime profile separately selects a target such as TypeScript, Python, or Rust; a provider pack; capability-to-provider mappings; and symbolic configuration or secret slots. The domain model does not change when PostgreSQL is exchanged for another compatible persistence provider. Runtime observations also stay separate: application commit, provider acceptance, delivery, consumer commit, and business completion do not imply one another.

STAGES 8–9

TDE and formal providers

DomainSpec identifies a requirement. TDE turns a supported requirement into neutral, reproducible test obligations. A provider executes one concrete version of those obligations.

The three transformations

1	**DomainSpec obligation:** "The SubmitOrder realization must apply OrderCanBeSubmitted before the state-changing commit."	what must hold
2	**TDE derivation:** generate neutral cases such as an empty order being rejected, a valid order being admitted, and no rejected case producing a commit or event.	what to check
3	**Provider execution:** translate those neutral cases into an exact TypeScript test, Python test, Rust test, solver query, or other admitted provider request.	what happened

Test Derivation Engine boundary

```
current FormalClaim
  + capability assessment
  + preservation-backed executable projection
  → provider-neutral obligations and bounded cases
```

A neutral obligation carries stable identities, normalized inputs, expected relations or oracle requirements, provenance, residue, and a claim ceiling. It deliberately contains no TypeScript, Python, Rust, Lean, SMT, Alloy, TLC, or Soufflé branch. This lets the same obligation be realized by more than one provider without turning provider syntax into business meaning.

Classification before execution

executable-bounded · **solver-admissible** · **proof-provider-required** · **unsupported** · **indeterminate**

Only an admitted executable-bounded slice enters ordinary test derivation. The theorem route, solver route, temporal model-checking route, relational route, and Datalog route remain distinct.

Provider evidence is exact

A provider-runtime binding identifies the provider version and digest, source artifact, command, environment allowlist, fixtures, resource limits, network policy, timeout, and supported result meanings. Raw output remains reachable.

TLA+/TLC route

A FormalClaim with temporal semantics is projected into a declared state model and properties. TLC can report behavior within the explored state space. It cannot establish correctness outside that model or show that implementation code preserves the model.

Lean route

An exact theorem statement and environment are checked through a theorem-provider request. A checked theorem is evidence about that model and binding—not proof of runtime correspondence.

Executable route

TDE derives cases; a selected runtime executes them; the result is bounded by the code, fixture, environment, and observation boundary.

Cross-provider discipline

Provider agreement is non-voting. Several weak results cannot vote themselves into a stronger claim; disagreement, unknown, missing, or stale evidence remains visible.

> **Fail-closed rule.** Unsupported, timeout, malformed, stale, unknown, provider error, and missing prerequisites never become PASS through aggregation.

STAGE 10

EVF: the evidence accountant

The Evolvable Validation Framework asks whether every declared path received the right kind of current evidence—and exposes exactly what remains unresolved.

```
For each reachable path p:
  desired residue ρ(p) exists
  owner(p) exists
  obligations O(p) are explicit
  required evidence kinds K(p) are explicit
  current evidence E(p) is digest-bound

supported(p) only if every obligation and required evidence kind
has current positive evidence, comparable under a non-inflating join,
with no contradiction, stale input, structural defect, or authority defect.
```

Evidence kinds are orthogonal

Evidence kind	Answers	Does not establish alone
Specification closure	Did we describe the declared paths and obligations?	Implementation or behavior
Formal proof	Does a proposition follow from an exact model?	Implementation preservation
Bounded exploration	Does a property hold in a finite declared state space?	Behavior outside the bound
Implementation test	What did code do for named inputs and environment?	Exhaustive or live-system behavior
Runtime observation	What happened in one named execution?	Universal or future correctness
Operational control	Did the system stop, retain, reconcile, alert, or escalate?	Internal code correctness

Typed outcomes

PASS_BOUNDED · NON_PASS · UNSUPPORTED · INDETERMINATE · STALE · MISSING · FAILED · BLOCKED

A positive conclusion is bound to the exact claim, projection, assumptions, provider, environment, inputs, result semantics, and source identities named by the join. EVF produces candidate-readiness evidence; it does not approve the product.

> Hype scales code. Rigor scales trust. The scarce resource is not output—it is justified confidence.

WORKED EXAMPLE

Order submission, end to end

A small example shows where meaning, formal reasoning, implementation, and evidence remain separate.

1. Clarify the language

Order is an identity-bearing Entity. **Order line** means a retained purchasable-item occurrence. **Non-negative total** is defined over the exact Money representation. Owners and unresolved questions are recorded.

2. Map business intent

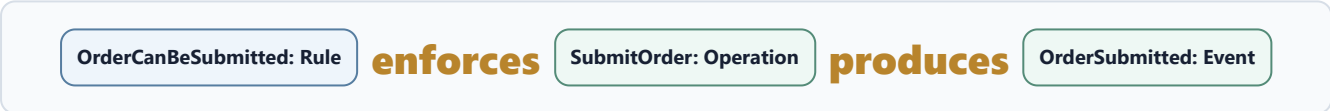


3. Bind one FormalClaim and application Rule

```
FormalClaim OrderCanBeSubmittedClaim
  proposition: ∀ o : Order · HasLine(o) ∧ NonNegative(Total(o))
  ceiling: submission eligibility only

Rule OrderCanBeSubmitted
  condition-bound-to OrderCanBeSubmittedClaim
  enforces SubmitOrder
```

4. Author and compile the system semantics



DomainSpec checks the endpoint types, resolves the source graph, derives applicable obligations, and emits deterministic projections and receipts.

5. Evaluate several different questions

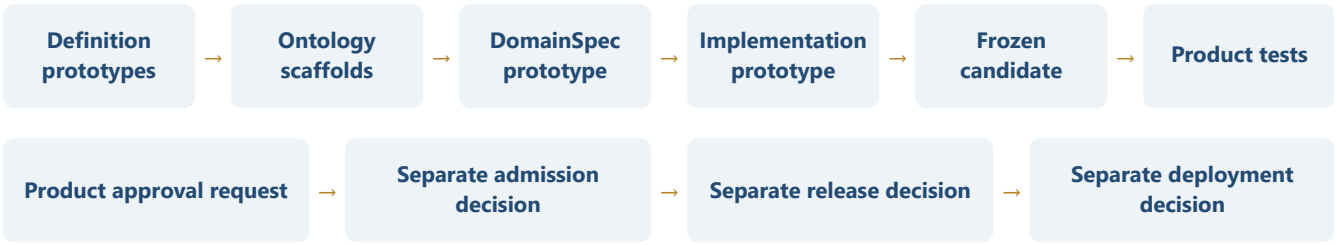
Question	Possible route	Honest conclusion
Can a submitted order ever have no lines in the temporal model?	TLA+/TLC	No counterexample within this exact model and explored space—or a concrete trace.
Does the formula follow from the encoded assumptions?	Lean or SMT	Bounded to the exact statement, theory, assumptions, and environment.
Does generated code call the Rule before committing?	Static observer + executable test	Observed for exact source, target, fixture, and tool versions.
Was the event accepted and consumed?	Runtime loopback observation	Observed at named commit, broker, delivery, and consumer boundaries only.

> **EVF conclusion example.** `PASS_BOUNDED` is possible only when the selected obligation's required evidence kinds are current and the exact join closes. It still does not mean "the production commerce system is correct."

CONSTRUCTION GOVERNANCE

D65: evidence does not authorize itself

CyberAlchemy surrounds the technical path with a prototype-first construction invariant. Learning is encouraged; authority is delayed until an exact tested candidate exists.



The invariant

```
asserted_strength(claim) ≤ strength(declared_support)
```

Support may be a source citation, validation, witness, accepted premise, owner decision, bounded model-checking result, or formal proof, depending on the claim. The rule does not demand certainty; it demands honest wording.

Present implementation posture

IMPLEMENTED — Canonical 13-type backend language; typed relationship checks; deterministic graph compilation; manifests and receipts; selected TypeScript, Python, and Rust realization evidence; substantial CLI, lowering, and provider-generation paths.

PARTIAL — AlchLang authoring-to-runtime composition; FormalClaim contracts and profile model; runtime/provider paths; system-ontology projections; bounded TDE/EVF candidate verticals and independent replay controls.

OPEN — One reconciled FormalClaim source contract across current TDE intake; the general TDE/formal-provider/EVF join; complete concise AlchLang surface; full production correspondence; external publication, release, and deployment decisions.

What the architecture claims

Yes

It is possible to preserve exact identities, provenance, typed relations, obligations, observations, and claim ceilings across a long software-construction chain.

No

The existence of that chain does not prove the business is right, every translation is complete, the implementation preserves every model, or a product should be released.

DISCUSSION INVITATION

Where formal-methods criticism is most valuable

The goal of this document is not to present a closed system. It is to make the seams precise enough to attack.

Semantic preservation

What evidence is sufficient to show that a projection preserves the business meaning selected by a FormalClaim?

Temporal adequacy

How should the system decide that a workflow claim genuinely requires a TLA+ route, rather than an executable or relational approximation?

Model–implementation correspondence

What is the strongest defensible bridge between a checked temporal model and an observed implementation?

Evidence joins

Which results are comparable, and what algebra prevents several weak observations from inflating into a stronger claim?

Change and invalidation

Which changes create a new FormalClaim, which only invalidate a projection, and which make prior evidence stale?

Human review

Can formal, plain-language, and domain-context views improve comprehension without becoming competing semantic authorities?

The proposed bridge is not “business prose → automatic proof.” It is “owned meaning → explicit formal subject → bounded providers → non-inflating evidence → separate decision.”

Technical source basis

- **CyberAlchemy definitions:** `authority/definitions/DEFINITIONS.md` — ontology, claim-proof bound, intent, and D65 construction invariant.
- **Define:** `define/README.md` and `define/business-ontology/README.md` — Definition Packs, dispositions, business ontology, and construction boundary.
- **End-to-end architecture:** `craft/projects/domainspec-v2/AUTHORING-TO-RUNTIME.md`.
- **DomainSpec product:** `PRODUCT-THESIS.md`, `ARCHITECTURE.md`, the DS-D1 meta-type registry, `definitions/meta-types/current-implementation.md`, and `impl/cli/README.md`.
- **Formalization:** `formalization/README.md`, `FORMALCLAIM-AND-PROFILES.md`, `UNDERSTANDING-PROJECTION.md`, and `TDE-EVF-INTEGRATION.md`.
- **EVF candidate definition:** `authority/definitions/decisions/evolvable-validation-framework-canonical-documentation/candidate/EVF.md`.

> **Status note.** CyberAlchemy and several described surfaces are private candidates under active construction. “Implemented” in this overview refers only to bounded repository evidence, not external release, production fitness, or authority.